第 1 批讲义：从反向传播到手写 GPT

大模型学习计划 · 阶段 0 · 第 1–2 周 · 对应 Karpathy《Neural Networks: Zero to Hero》前 7 讲

怎么用这份讲义

- 这份讲义替代视频，内容按"直觉 → 公式 → 代码 → 常见坑"展开。所有代码都已实际跑通。
- **代码请自己一行行敲**，不要复制粘贴，也不要让 agent 代写。环境（Python、PyTorch）可以交给 agent。
- 数据集两个，让 agent 下载即可：
  - `names.txt`（3.2 万个英文名）：`https://raw.githubusercontent.com/karpathy/makemore/master/names.txt`
  - `input.txt`（莎士比亚文本，约 1MB）：`https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt`
- 节奏：第 1 周读第 1–3 讲，第 2 周读第 4–5 讲。

第 1 讲　反向传播：手写一个自动求导引擎（micrograd）

1.1 导数是什么

导数回答一个问题：**把输入轻轻推一下，输出会变多少？** 对 $f(x)=3x^2-4x+5$，在 $x=3$ 处把 x 加 $h=0.0001$，f 增加约 $14h$，所以 $f'(3)=14$。神经网络训练就是：算出"损失对每个参数的导数"（梯度），然后把每个参数往让损失变小的方向推一点。

1.2 计算图与链式法则

任何表达式都可以拆成一张图：节点是数值，边是运算。例如 $L = (a \cdot b + c) \cdot f$。

反向传播从输出往回走，每个节点的梯度 = **上游传来的梯度 × 本地导数**（链式法则）。只需要记住几条本地规则：

运算	本地导数	直觉
$out = a + b$	$\partial out/\partial a = 1$	加法把梯度**原样分发**给两边
$out = a \cdot b$	$\partial out/\partial a = b$	乘法把梯度**乘以对方**的值
$out = a^k$	$k \cdot a^{k-1}$	幂函数求导
$out = \tanh(a)$	$1 - out^2$	激活函数，输出在 $(-1, 1)$

两个关键细节：

1. **按拓扑序反向执行**：一个节点的梯度必须等它所有"下游使用者"都传回来之后才完整。所以先对图做拓扑排序，再倒序调用每个节点的 `_backward`。
2. **梯度要累加（`+=`），不能覆盖（`=`）**：同一个变量被用了多次（比如 b = a + a），梯度要从每条路径加起来。这也是为什么每轮训练前必须把梯度**清零**。

1.3 代码：Value 类 + 一个小 MLP

```
import math, random

class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')
        def _backward():
            self.grad += out.grad          # 加法：梯度原样分发
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')
        def _backward():
            self.grad += other.data * out.grad   # 乘法：梯度乘以"对方"
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def __pow__(self, k):
        out = Value(self.data ** k, (self,), f'**{k}')
        def _backward():
            self.grad += k * self.data ** (k - 1) * out.grad
        out._backward = _backward
        return out

    def tanh(self):
        t = math.tanh(self.data)
        out = Value(t, (self,), 'tanh')
        def _backward():
            self.grad += (1 - t ** 2) * out.grad
        out._backward = _backward
        return out

    def backward(self):
        topo, visited = [], set()
        def build(v):
            if v not in visited:
                visited.add(v)
                for c in v._prev:
                    build(c)
                topo.append(v)
        build(self)
        self.grad = 1.0
        for v in reversed(topo):
            v._backward()
```

```
    def __neg__(self): return self * -1
    def __sub__(self, other): return self + (-other)
    def __radd__(self, other): return self + other
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other ** -1

class Neuron:
    def __init__(self, nin):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(random.uniform(-1, 1))
    def __call__(self, x):
        return sum((wi * xi for wi, xi in zip(self.w, x)), self.b).tanh()
    def parameters(self): return self.w + [self.b]

class Layer:
    def __init__(self, nin, nout):
        self.neurons = [Neuron(nin) for _ in range(nout)]
    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs
    def parameters(self): return [p for n in self.neurons for p in n.parameters()]

class MLP:
    def __init__(self, nin, nouts):
        sz = [nin] + nouts
        self.layers = [Layer(sz[i], sz[i + 1]) for i in range(len(nouts))]
    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x
    def parameters(self): return [p for l in self.layers for p in l.parameters()]

if __name__ == '__main__':
    random.seed(42)
    xs = [[2.0, 3.0, -1.0], [3.0, -1.0, 0.5], [0.5, 1.0, 1.0], [1.0, 1.0, -1.0]]
    ys = [1.0, -1.0, -1.0, 1.0]
    model = MLP(3, [4, 4, 1])
    for step in range(100):
        ypred = [model(x) for x in xs]
        loss = sum((yp - yt) ** 2 for yp, yt in zip(ypred, ys))
        for p in model.parameters():
            p.grad = 0.0                    # 清零！否则梯度会累加
        loss.backward()
        for p in model.parameters():
            p.data -= 0.05 * p.grad
        if step % 20 == 0:
            print(step, round(loss.data, 4))
    print('final', round(loss.data, 4), [round(model(x).data, 2) for x in xs])
```

运行后 loss 从 5.23 降到约 0.01，四个预测值逼近目标 [1, −1, −1, 1]。

1.4 从标量到 PyTorch

micrograd 的每个 Value 是一个标量；PyTorch 做的是同一件事，只是每个节点是**张量**（一整块数），因此能在 GPU 上并行。`loss.backward()`、`p.grad`、`requires_grad` 的语义和这里完全一致。

1.5 常见坑

- 忘记清零梯度 → 梯度跨轮累加，训练发散。
- 学习率太大 → loss 来回震荡甚至变成 nan；太小 → 下降极慢。

第 2 讲　语言建模入门：Bigram 模型（makemore 第 1 部分）

2.1 任务

给定 3.2 万个名字，训练一个模型**生成像名字的新词**。这就是语言模型的本质：**给定前文，预测下一个字符（token）的概率分布**，然后反复采样。

字符表：26 个字母 + 一个特殊符号 `.`（同时代表开头和结尾），共 27 个。

2.2 方法一：直接数

Bigram 只看前一个字符。统计所有"字符对"出现的次数，得到 27×27 的计数矩阵 N，每行归一化就得到条件概率 P(下一个 | 当前)。

2.3 如何评价模型：负对数似然（NLL）

模型好坏 = 它给真实数据分配的概率有多高。把所有真实字符对的概率取对数、相加、取负、求平均：

NLL = −(1/n) Σ log P(真实下一个字符)

越小越好。完全瞎猜（均匀分布）时 NLL = −ln(1/27) ≈ **3.29**，记住这个数，后面会反复用到。Bigram 计数法能做到约 **2.45**。

平滑：某些字符对从没出现过，概率为 0，log 0 = −∞。给所有计数 +1 即可避免。

2.4 方法二：用神经网络做同一件事

1. 输入字符变成 one-hot 向量（27 维，只有一位是 1）。
2. 乘一个 27×27 的权重矩阵 W，得到 27 个 **logits**（可以理解为"对数计数"）。
3. **softmax**：exp 后归一化，变成概率分布。
4. 损失 = 交叉熵 = 上面的 NLL。梯度下降优化 W。

训练后 loss 收敛到和计数法几乎一样（约 2.45–2.49），说明两者是同一个模型。好处是：**神经网络这条路可以无限扩展**，而计数表在上下文变长时会爆炸。

小知识：one-hot 乘矩阵其实就是"查表取 W 的某一行"，这就是后来的 **Embedding 层**。给 W 加 L2 正则，效果等价于计数法里的平滑。

2.5 代码

```
import torch
import torch.nn.functional as F

words = open('names.txt').read().splitlines()
chars = sorted(set(''.join(words)))
stoi = {s: i + 1 for i, s in enumerate(chars)}
stoi['.'] = 0                                   # '.' 同时表示开头和结尾
itos = {i: s for s, i in stoi.items()}
```

```python
# --- 方法一：计数 ---
N = torch.zeros((27, 27), dtype=torch.int32)
for w in words:
    cs = ['.'] + list(w) + ['.']
    for a, b in zip(cs, cs[1:]):
        N[stoi[a], stoi[b]] += 1
P = (N + 1).float()                              # +1 平滑，避免概率为 0
P /= P.sum(1, keepdim=True)

nll, n = 0.0, 0
for w in words:
    cs = ['.'] + list(w) + ['.']
    for a, b in zip(cs, cs[1:]):
        nll -= torch.log(P[stoi[a], stoi[b]]); n += 1
print('计数法 平均NLL:', (nll / n).item())

# --- 方法二：神经网络（一层线性 + softmax）---
xs, ys = [], []
for w in words:
    cs = ['.'] + list(w) + ['.']
    for a, b in zip(cs, cs[1:]):
        xs.append(stoi[a]); ys.append(stoi[b])
xs, ys = torch.tensor(xs), torch.tensor(ys)
g = torch.Generator().manual_seed(2147483647)
W = torch.randn((27, 27), generator=g, requires_grad=True)
for k in range(100):
    logits = F.one_hot(xs, 27).float() @ W     # 等价于 W[xs]
    loss = F.cross_entropy(logits, ys) + 0.01 * (W ** 2).mean()
    W.grad = None
    loss.backward()
    W.data += -50 * W.grad
print('神经网络 loss:', loss.item())

# --- 采样 ---
for _ in range(5):
    out, ix = [], 0
    while True:
        ix = torch.multinomial(P[ix], 1, generator=g).item()
        if ix == 0: break
        out.append(itos[ix])
    print(''.join(out))
```

生成结果大多不像名字，因为只看一个字符的上下文太短了。

第 3 讲　多层感知机语言模型（makemore 第 2 部分）

3.1 思路（Bengio 等 2003 年的论文）

- 上下文扩大到前 **3 个**字符（`block_size = 3`）。
- 每个字符查表得到一个低维向量（**embedding**，这里 10 维）。相似的字符在训练后会在向量空间里靠近。
- 3 个向量拼接 → 隐藏层（tanh）→ 输出 27 个 logits → softmax。

3.2 训练工程的基本功

- **Minibatch**：每步只随机取 32 个样本算梯度。梯度不精确，但快几百倍，整体更划算。
- **`F.cross_entropy`**：别自己写 exp/归一化，它更快、数值更稳定（大 logits 不会溢出）。

- **找学习率**：从极小到较大扫一遍，看 loss 在哪个区间下降最快；训练后期再把学习率降 10 倍（衰减）。
- **数据集三分**：训练集 80%（训练参数）、验证集 10%（调超参）、测试集 10%（最后只看一次）。
- **欠拟合 vs 过拟合**：train loss ≈ dev loss 说明欠拟合，可以把模型做大；train 远低于 dev 说明过拟合，模型在背数据。

3.3 代码

```
import random
import torch
import torch.nn.functional as F

words = open('names.txt').read().splitlines()
chars = sorted(set(''.join(words)))
stoi = {s: i + 1 for i, s in enumerate(chars)}; stoi['.'] = 0
itos = {i: s for s, i in stoi.items()}
block_size = 3                                  # 用前 3 个字符预测下一个

def build_dataset(ws):
    X, Y = [], []
    for w in ws:
        ctx = [0] * block_size
        for ch in w + '.':
            X.append(ctx); Y.append(stoi[ch])
            ctx = ctx[1:] + [stoi[ch]]
    return torch.tensor(X), torch.tensor(Y)

random.seed(42); random.shuffle(words)
n1, n2 = int(0.8 * len(words)), int(0.9 * len(words))
Xtr, Ytr = build_dataset(words[:n1])            # 80% 训练
Xdev, Ydev = build_dataset(words[n1:n2])        # 10% 验证（调超参）
Xte, Yte = build_dataset(words[n2:])            # 10% 测试（最后才看）

g = torch.Generator().manual_seed(2147483647)
n_embd, n_hidden = 10, 200
C  = torch.randn((27, n_embd), generator=g)
W1 = torch.randn((n_embd * block_size, n_hidden), generator=g) * (5/3) / (n_embd * block_size)
** 0.5
b1 = torch.randn(n_hidden, generator=g) * 0.01
W2 = torch.randn((n_hidden, 27), generator=g) * 0.01   # 输出层缩小 → 初始 loss≈3.29
b2 = torch.zeros(27)
params = [C, W1, b1, W2, b2]
for p in params: p.requires_grad = True

def forward(X):
    emb = C[X].view(X.shape[0], -1)             # (B, 3*n_embd)
    h = torch.tanh(emb @ W1 + b1)               # (B, n_hidden)
    return h @ W2 + b2                          # (B, 27)

steps = 50000
for i in range(steps):
    ix = torch.randint(0, Xtr.shape[0], (32,), generator=g)   # minibatch
    loss = F.cross_entropy(forward(Xtr[ix]), Ytr[ix])
    for p in params: p.grad = None
    loss.backward()
    lr = 0.1 if i < steps // 2 else 0.01        # 学习率衰减
    for p in params: p.data += -lr * p.grad
    if i % 10000 == 0: print(i, round(loss.item(), 4))

with torch.no_grad():
    print('train', F.cross_entropy(forward(Xtr), Ytr).item())
    print('dev  ', F.cross_entropy(forward(Xdev), Ydev).item())
    for _ in range(5):
        ctx, out = [0] * block_size, []
        while True:
```

```
            probs = F.softmax(forward(torch.tensor([ctx])), dim=1)
            ix = torch.multinomial(probs, 1, generator=g).item()
            ctx = ctx[1:] + [ix]
            if ix == 0: break
            out.append(itos[ix])
        print(''.join(out))
```

结果：train ≈ 2.10，dev ≈ 2.14，明显好于 bigram，生成的名字已经像样（如 "krissa"、"yodarya"）。代码里已经加了第 4 讲的两个初始化技巧。

第 4 讲　激活、梯度与 BatchNorm（makemore 第 3 部分）

深网络为什么难训？核心是**数值的尺度**在层层传递中失控。

4.1 检查初始 loss

初始化时模型应该"一无所知"，即输出接近均匀分布，loss 应约等于 **3.29**。如果一开始是 27 之类的大数，说明 logits 太大、模型"自信地乱猜"，前几千步都浪费在把 logits 压小上。

修正：输出层权重乘 0.01，偏置设 0。

4.2 tanh 饱和

如果隐藏层的输入太大，tanh 的输出大量贴在 ±1。此时本地导数 $1 - t^2 \approx 0$，**梯度传不回去**，神经元"死掉"。诊断方法：画隐藏层激活的直方图，看有多少值的绝对值大于 0.99。

4.3 Kaiming 初始化

让每层输出的方差保持在 1 左右：权重 = randn × **gain** / $\sqrt{\textbf{fan_in}}$。fan_in 是输入维度，tanh 的 gain 取 5/3，ReLU 取 $\sqrt{2}$。PyTorch 里是 `torch.nn.init.kaiming_normal_`。

4.4 BatchNorm

与其小心初始化，不如**直接把每层的输入标准化**：

y = γ · (x − 批均值) / $\sqrt{(批方差 + \varepsilon)}$ + β

- γ、β 是可学习参数，让网络在需要时恢复原来的分布。
- 训练时用当前批的统计量；同时维护一个滑动平均（running mean/var），**推理时用滑动平均**。
- BatchNorm 前面那层的偏置是多余的（会被减均值抵消掉）。
- 副作用：同一批样本相互影响。后来的 Transformer 改用 **LayerNorm**（对单个样本的特征维度做标准化），没有这个问题。

4.5 训练健康诊断清单

1. 初始 loss 是否接近理论值。

2. 各层激活分布：标准差在 1 附近、饱和比例低。
3. 各层梯度分布：不同层的梯度量级相近。
4. **更新量 / 参数值** 的比值：大约 1e-3 比较健康。远小于说明学习率太低，远大于说明太高。

（选做：makemore 第 4 部分是不用 autograd、手推整个网络的反向传播，适合想彻底吃透梯度的人。）

第 5 讲　从零写一个 GPT

5.1 任务与数据

用莎士比亚全集（约 100 万字符，65 种字符）训练字符级语言模型。前 90% 训练、后 10% 验证。

训练样本：随机截取长度为 `block_size` 的片段 x，目标 y 是 x 右移一位。**一个片段同时包含 block_size 个训练样例**：用第 1 个字符预测第 2 个，用前 2 个预测第 3 个……，所以模型学会了处理从 1 到 block_size 的任意长度上下文。

5.2 自注意力的"数学技巧"

我们希望每个位置能**汇总它之前所有位置的信息**。最简单的做法是对前文取平均。用矩阵乘法实现：

1. 一个 T×T 的下三角矩阵（上三角为 −∞，下三角为 0）。
2. 每行做 softmax → 变成每行"对前面位置均匀加权"的矩阵。
3. 用它乘以序列 → 每个位置得到前文的平均。

自注意力就是把这个"均匀权重"换成**由数据决定的权重**。

5.3 Q、K、V

每个位置的向量 x 经过三个线性层，得到：

- **Query（查询）**：我在找什么信息？
- **Key（键）**：我包含什么信息？
- **Value（值）**：如果你关注我，我给你什么？

位置 i 对位置 j 的注意力分数 = q_i · k_j。分数矩阵经过**因果掩码**（未来位置设为 −∞）和 softmax 后，对 V 加权求和。

为什么除以 $\sqrt{\text{head_size}}$：q·k 的方差会随维度增长，值太大时 softmax 会变得极其尖锐（接近 one-hot），每个位置只盯一个位置看，梯度也变差。缩放后方差回到 1 左右。

几个要点：

- 注意力本身**没有位置概念**，它只是对一组向量做加权求和，所以要额外加位置嵌入。
- 同一批里的不同样本之间**不会**互相注意。

- "自"注意力是因为 Q、K、V 都来自同一个 x；交叉注意力中 K、V 来自另一个来源（例如翻译里的原文）。
- 编码器（如 BERT）没有因果掩码；GPT 是**纯解码器**，有掩码。

5.4 组装 Transformer Block

- **多头注意力**：并行跑多个小头（例如 6 个 64 维），拼接后再投影。不同头可以关注不同的关系。
- **前馈网络（FFN）**：每个位置独立做"线性 → ReLU → 线性"，中间层放大 4 倍。注意力负责**通信**，FFN 负责**计算/思考**。
- **残差连接**：x = x + 子层(x)。梯度可以沿着"加法高速公路"直通底层，深网络才训得动。
- **LayerNorm**：现代做法是 Pre-LN，即在进入子层之前做归一化；最后输出前再加一个。
- **Dropout**：训练时随机丢弃部分激活，防止过拟合。

5.5 完整代码

```
import sys
import torch
import torch.nn as nn
from torch.nn import functional as F

# ---- 超参（GPU 配置；CPU 上把注释里的小配置换进来）----
batch_size, block_size = 64, 256     # CPU: 32, 64
n_embd, n_head, n_layer = 384, 6, 6  # CPU: 64, 4, 4
dropout, lr = 0.2, 3e-4              # CPU: 0.0, 1e-3
max_iters, eval_interval, eval_iters = 5000, 500, 200
if len(sys.argv) > 1 and sys.argv[1] == 'cpu':
    batch_size, block_size, n_embd, n_head, n_layer, dropout, lr = 32, 64, 64, 4, 4, 0.0, 1e-3
    max_iters, eval_iters = int(sys.argv[2]) if len(sys.argv) > 2 else 3000, 50
device = 'cuda' if torch.cuda.is_available() else 'cpu'
torch.manual_seed(1337)

text = open('input.txt', encoding='utf-8').read()
chars = sorted(set(text)); vocab_size = len(chars)          # 65
stoi = {c: i for i, c in enumerate(chars)}; itos = {i: c for i, c in enumerate(chars)}
encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join(itos[i] for i in l)
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data)); train_data, val_data = data[:n], data[n:]

def get_batch(split):
    d = train_data if split == 'train' else val_data
    ix = torch.randint(len(d) - block_size, (batch_size,))
    x = torch.stack([d[i:i + block_size] for i in ix])
    y = torch.stack([d[i + 1:i + block_size + 1] for i in ix])   # 目标 = 右移一位
    return x.to(device), y.to(device)

@torch.no_grad()
def estimate_loss():
    model.eval(); out = {}
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            _, loss = model(*get_batch(split)); losses[k] = loss.item()
        out[split] = losses.mean().item()
    model.train(); return out

class Head(nn.Module):
    """单头因果自注意力"""
    def __init__(self, head_size):
        super().__init__()
```

```
        self.key   = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)
    def forward(self, x):
        B, T, C = x.shape
        k, q, v = self.key(x), self.query(x), self.value(x)            # (B,T,hs)
        wei = q @ k.transpose(-2, -1) * k.shape[-1] ** -0.5             # (B,T,T) 缩放点积
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))    # 因果掩码
        wei = self.dropout(F.softmax(wei, dim=-1))
        return wei @ v                                                  # (B,T,hs)

class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(head_size * num_heads, n_embd)
        self.dropout = nn.Dropout(dropout)
    def forward(self, x):
        return self.dropout(self.proj(torch.cat([h(x) for h in self.heads], dim=-1)))

class FeedForward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_embd, 4 * n_embd), nn.ReLU(),
                                 nn.Linear(4 * n_embd, n_embd), nn.Dropout(dropout))
    def forward(self, x): return self.net(x)

class Block(nn.Module):
    """通信（注意力）+ 计算（FFN），Pre-LN + 残差"""
    def __init__(self, n_embd, n_head):
        super().__init__()
        self.sa = MultiHeadAttention(n_head, n_embd // n_head)
        self.ffwd = FeedForward(n_embd)
        self.ln1, self.ln2 = nn.LayerNorm(n_embd), nn.LayerNorm(n_embd)
    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

class GPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, n_embd)
        self.pos_emb = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)
    def forward(self, idx, targets=None):
        B, T = idx.shape
        x = self.tok_emb(idx) + self.pos_emb(torch.arange(T, device=idx.device))
        logits = self.lm_head(self.ln_f(self.blocks(x)))                # (B,T,vocab)
        loss = None if targets is None else F.cross_entropy(
            logits.view(B * T, -1), targets.view(B * T))
        return logits, loss
    @torch.no_grad()
    def generate(self, idx, max_new_tokens):
        for _ in range(max_new_tokens):
            logits, _ = self(idx[:, -block_size:])                      # 截断到上下文长度
            probs = F.softmax(logits[:, -1, :], dim=-1)
            idx = torch.cat([idx, torch.multinomial(probs, 1)], dim=1)
        return idx

model = GPT().to(device)
print(sum(p.numel() for p in model.parameters()) / 1e6, 'M 参数')
opt = torch.optim.AdamW(model.parameters(), lr=lr)
for it in range(max_iters + 1):
    if it % eval_interval == 0:
        l = estimate_loss(); print(f"step {it}: train {l['train']:.4f}, val {l['val']:.4f}")
    _, loss = model(*get_batch('train'))
```

```
    opt.zero_grad(set_to_none=True); loss.backward(); opt.step()
print(decode(model.generate(torch.zeros((1, 1), dtype=torch.long, device=device), 300)[0].toli
st()))
```

运行方式：有 GPU 直接 `python gpt.py`（约 1080 万参数）；只有 CPU 用 `python gpt.py cpu 3000`（约 21 万参数的小配置）。

5.6 你应该看到的结果

配置	参数量	验证集 loss	生成效果
bigram 基线	—	约 2.5	乱码
CPU 小配置 3000 步	0.21M	约 1.78	有剧本格式，单词半对半错
GPU 配置 5000 步	10.8M	约 1.48（Karpathy 报告值）	像模像样的莎士比亚风格，语义不通

5.7 从这个 GPT 到 ChatGPT

- **预训练**：就是我们做的事，只是数据是互联网规模、token 是子词而非字符、模型大几万倍。得到的是一个"文档补全器"，不会对话。
- **后训练**：用问答数据做监督微调，再用人类偏好做强化学习（RLHF），把补全器变成助手。这是阶段 4 的内容。

完成标准（做完再找我）

- ☐ 不看讲义，凭记忆重写 micrograd 并训练出小 MLP。
- ☐ 自己敲完 GPT，跑出上表对应配置的验证 loss，并能生成文本。
- ☐ 用自己的话写清楚三件事：
    1. 自注意力里 Q、K、V 各干什么；
    2. 为什么要因果掩码；
    3. 残差连接和 LayerNorm 各解决了什么问题。

把代码、loss 曲线或卡住的地方发我，我看过之后给第 2 批（前沿模型架构）。

延伸阅读（文字版，可选）

- Karpathy 的 micrograd、makemore、nanoGPT 三个 GitHub 仓库（README 和代码本身就是很好的教材）
- Jay Alammar《The Illustrated Transformer》（图解）
- 论文：Bengio 等《A Neural Probabilistic Language Model》（2003）；Vaswani 等《Attention Is All You Need》（2017）